

	Style	Concept	# Instances
Single Object	Filled or Outlined	Circle	10K
		Ellipse	10K
		Rectangle	10K
		Triangle	10K
		Polygon	20K
		Line Segment	10K
		Grid	10K
		Path	10K
		Graph	10K
Composition	Filled	Circle	5K
		Rectangle	5K
		Triangle	5K
		Line Segment	5K
	Outlined	Circle	10K
		Rectangle	10K
		Triangle	10K
		Line Segment	10K
Total			160K

Table 4: PVD 160K dataset statistics.

- **Ellipse:** Randomly sample a center, a major axis, and a minor axis, then randomly rotate by an angle. Verify if the ellipse is largely within the canvas; if not, try again.
- **Rectangle:** Randomly sample a top-left corner, a width, and a height, then randomly rotate by an angle. Verify if the rectangle is largely within the canvas; if not, try again.
- **Triangle:** Randomly sample three points as vertices to draw a triangle. Check if the area is larger than a threshold; if not, try again.
- **Polygon:** Randomly sample $N \in [5, 20]$ points. Order the points with respect to the centroid so that no intersections will happen when connected with a polyline. Draw a polygon with the sampled points. Check if the polygon has an area larger than a threshold; if not, try again.
- **Path:** Randomly and iteratively sample $N \in [3, 16]$ points, connect the newly sampled point with the previous point to form a line segment. Verify if the newly added line segment does not intersect with any of the previous line segments; if yes, resample the point.
- **Grid:** Sample a grid of points with a size $M \times N$ where $M, N \in [2, 6]$. First, use Depth First Search (DFS) algorithm to connect all grid vertices into a connected graph. Then randomly add additional edges between adjacent vertices.
- **Graph:** Randomly sample $N \in [4, 16]$ points. First, use Kruskal’s algorithm (Kruskal, 1956) to find a Minimum Spanning Tree that connects all the points. Then randomly add additional edges to the graph.

Generation procedures (Composition): Iteratively draw shapes on the canvas chosen from the following set of object types: [“circle”, “rectangle”, “triangle”, “line segment”]. After the first shape is drawn, at each iteration, the later shapes are constrained to have the same color as the previous shapes. We ensure overlap between the newly added shape and the previous shapes, while making sure that the intersection ratio does not exceed a predefined threshold. This prevents cases where one shape entirely contains another, making it impossible to decode into individual Primal Visual Description elements.

PVD 160K dataset: Using the aforementioned generation procedure, we generate a large-scale dataset containing 160K (SVG, PVD) pairs for training the LLM-based SVG-to-PVD model. The detailed configuration can be found in Table 4.

	SSIM	DINOv2 Score	CLIP Score
w/o aug	0.892	0.874	0.886
w/ aug	0.895	0.893	0.893

Table 5: Impact of the data augmentation (Gaussian Blur and Pixel Noise detailed in §2.2) on SVG-to-PVD model perception performance.

Data augmentation details: To enhance the robustness of the SVG-to-PVD model to images with various sizes and quality, we introduce the following randomized data augmentation during data generation.

- **Random pixel noise:** Probability (how often to apply the augmentation): 0.1; Ratio range (what portion of the selected area will be filled with noise pixels): (0.01, 0.05); Intensity range (the intensity of the noise pixels): (0.1, 1.0); Dilate range (how many pixels will the selection area be extended from the boundary): (1, 3) in pixels; Noise size: (1, 3) in pixels.
- **Gaussian blur:** Probability (how often to apply the augmentation): 0.1; Radius: (0.1, 0.5).

Table 5 shows the ablation study with and without the data augmentations.

D Image-to-SVG Visual Encoding Quality

VTracer Encoding Quality. As introduced in Section 2.1, the first step of our proposed perception module is to encode a raster image into an SVG representation using VTracer, a rule-based converter that we found yields near-perfect reconstructions for vector-graphic-style inputs. To verify this, we convert the VTracer-encoded SVGs back into PNG images and compare them with the originals. The results are presented in Table 6.

Task	SSIM	DINOv2 Score	CLIP Score
Angle Classification	0.990	0.991	0.997
Length Comparison	0.994	0.970	0.986
Shapeworld Spatial 2Obj	0.997	0.984	0.995
Shapeworld Spatial mObj	0.991	0.984	0.988
Shapeworld Superlative	0.991	0.982	0.987
NLVR	0.994	0.989	0.993
Geoclidian	0.648	0.994	0.995
Maze 2×2	0.998	0.906	0.775
Maze 3×3	0.998	0.998	0.989
ALL	0.956	0.978	0.967

Table 6: VTracer SVG encoding reconstruction metrics. The exceptionally low SSIM score on Geoclidian is due to the fact that most pixels are transparent. Since we compute SSIM only on non-transparent regions, the score becomes more sensitive to slight pixel differences. However, the reconstruction appears visually near-perfect across all tasks, as reflected by the high DINOv2 score.

We observe near-perfect reconstruction quality from VTracer. For reference, the average reconstruction quality from the PVD representation (after SVG-to-PVD conversion) is SSIM = 0.895, DINOv2 Score = 0.880, and CLIP Score = 0.893, indicating that most perception errors arise during the SVG-to-PVD step. However, as discussed in Section 2.2, although SVG preserves all low-level features, it is extremely verbose and noisy—making it unintelligible to LLMs and LMMs. This observation motivates the development of the intermediate PVD representation.

Impact of VTracer Error to the End-task Performance. To further investigate whether imperfections in SVG encoding significantly affect end-task performance. We sort the instances for each task with respect to VTracer reconstruction quality, using the aggregated metric: SSIM + DINOv2 score + CLIP score. The

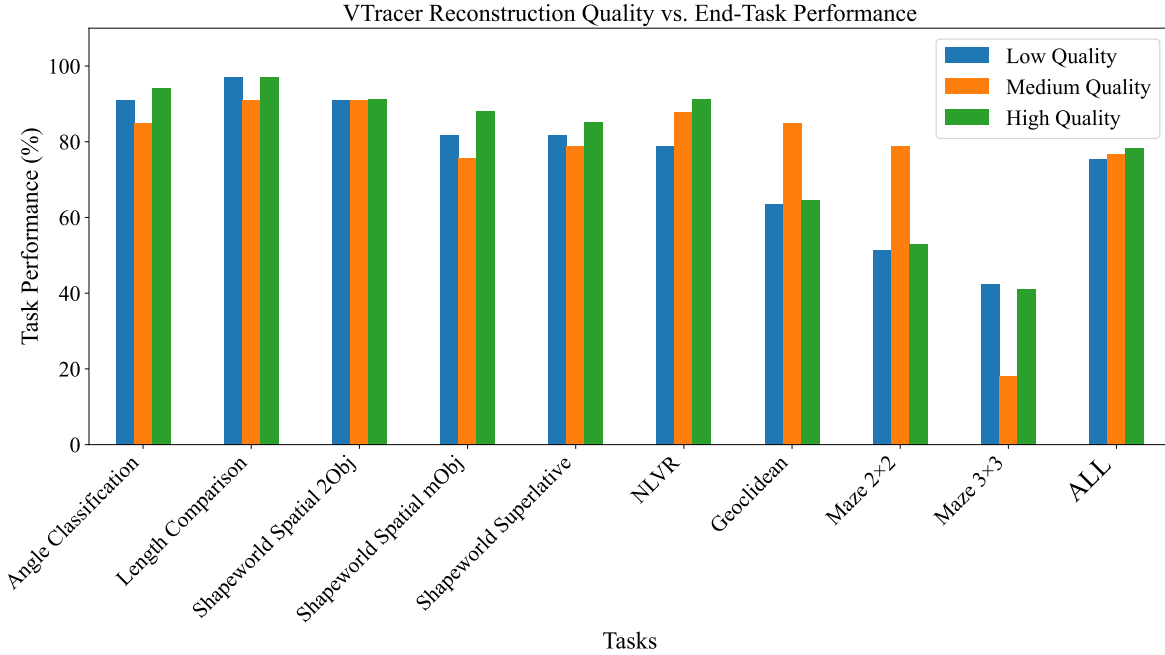


Figure 15: Impact of SVG encoding quality on end-task performance.

instances are then grouped into three bins corresponding to low, medium, and high VTracer quality. We examine the end-task performance (VDLM-mm + GPT-4o) of these groups to investigate whether performance is sensitive to VTracer errors. In Figure 15, we show the average accuracy on each task for each bin. Note that each bin has about 33 instances.

Overall, we observe that end-task performance is fairly robust to vectorization errors, while also exhibiting a trend similar to what we reported in Section 4.1, i.e., a positive correlation between perception quality and end-task performance.

E Additional Ablations on PVD Perception

PVD Model	SSIM $\uparrow$	DINOv2 Score $\uparrow$	CLIP Score $\uparrow$
SVG-to-PVD (Mistral-7B)	0.895	0.880	0.893
SVG-to-PVD (Qwen2.5-7B)	0.889	0.876	0.880

Table 7: Perception metrics of different SVG-to-PVD models.

Reasoner	PVD Model	AC	LC	SW-S 2Obj	SW-S mObj	SW Sup	NLVR	Geo	Maze 2x2	Maze 3x3	All
GPT-4o	–	0.63	0.57	0.97	0.82	0.92	0.81	0.71	0.46	0.08	0.663
GPT-4o	Mistral-7B	0.90	0.95	0.91	0.82	0.82	0.86	0.71	0.61	0.34	0.769
GPT-4o	Qwen2.5-7B	0.82	0.98	0.99	0.79	0.89	0.83	0.68	0.70	0.29	0.774

Table 8: End-task performance with different SVG-to-PVD models.

Different LLM choices for SVG-to-PVD model. We investigate the effect of using a different LLM for SVG-to-PVD translation. Specifically, we train a Qwen-2.5-7B[¶] base model on the PVD-160k dataset using the same number of epochs as our original model. We observe comparable perception and end-task performance to our original PVD model, which is based on Mistral-7B. Table 7 and Table 8 report the perception scores and end-task performance of SVG-to-PVD models trained with different LLM backbones. These results suggest that the SVG-to-PVD translation process is robust to the choice of LLM.

[¶]<https://huggingface.co/Qwen/Qwen2.5-7B>